Experimento: sincronización de luciérnagas con ESP32 en QEMU

Oct 23, 2026 · @Juan Ignacio Cleri

Idea general

Simulamos 8 ESP32 en QEMU, cada uno con hasta 4 LEDs. Cada LED es un oscilador que parpadea a su ritmo y se ajusta a los demás mediante mensajes MQTT con payload en Protocol Buffers. Un frontend muestra en vivo el parpadeo, la fase, los paquetes y las métricas, y variamos la configuración para comparar resultados.

Preguntas de investigación

1. ¿Pueden los nodos sincronizarse usando solo intercambio de mensajes entre pares por MQTT, sin un coordinador que calcule fases?
2. ¿Cómo afectan la cantidad de nodos, los LEDs por nodo, el acoplamiento, la pérdida, la latencia y el QoS a esa sincronización?
3. ¿Cuánto cuesta en mensajes, bytes y latencia la sincronización con protobuf sobre MQTT?

Estas preguntas corresponden a los objetivos de la monografía: evaluar MQTT y Protocol Buffers en capa 7 para redes inestables y con ancho de banda limitado.

Modelo: osciladores acoplados (Kuramoto)

Cada LED *i* tiene una fase θ_i que avanza con su frecuencia natural ω_i. Al llegar a 2π, el LED dispara y publica un flash, y reinicia su fase. Al recibir el flash de otro LED *j*, adelanta su fase en una cantidad proporcional al acoplamiento ε, y así tiende a alinearse con *j*.

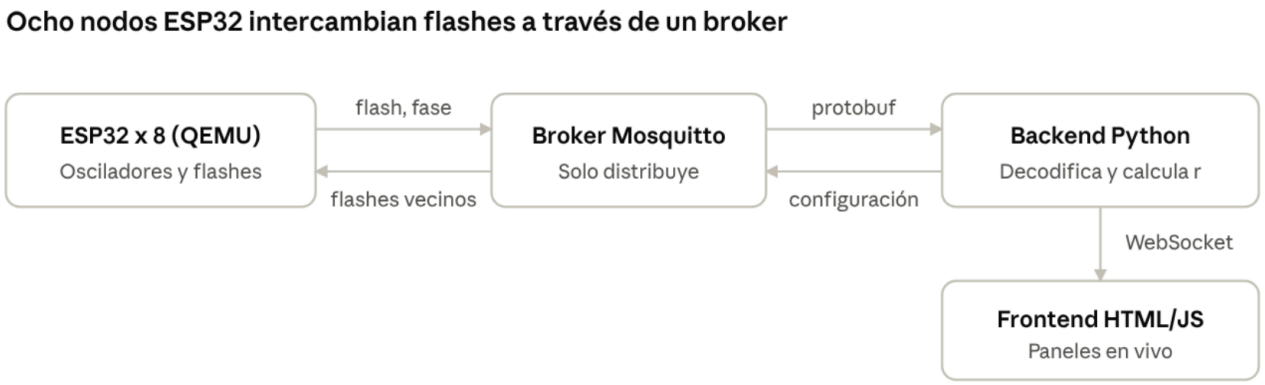
La sincronización se mide con el orden de fase de Kuramoto:

$$r = \left| \frac{1}{M} \sum_{j=1}^{M} e^{i\theta_j} \right|$$

con M igual al número total de LEDs. Un valor r cercano a 0 indica desorden y r cercano a 1 indica sincronía. Se considera convergida la red cuando $r \geq 0{,}95$.

Cada LED es un oscilador independiente, aunque comparta ESP con otros LEDs. Con 8 ESP de 4 LEDs hay hasta 32 osciladores.

Arquitectura



arquitectura · 8 nodos, 1 broker, backend y frontend

Los nodos no se comunican entre sí de forma directa: publican y reciben todo a través del broker. El backend observa el tráfico, calcula las métricas y envía la configuración.

Mensajes y tópicos

Los mensajes se definen en un único archivo `.proto` compartido por el firmware y por el backend. Ejemplo:

```
syntax = "proto3";

message Flash {
  uint32 esp_id = 1;
  uint32 led_id = 2;
  uint32 seq    = 3;
  uint64 t_us   = 4;
}

message PhaseReport {
  uint32 esp_id = 1;
  uint32 led_id = 2;
  float  phase  = 3;
  bool   on     = 4;
  uint64 t_us   = 5;
}

message RxEvent {
  uint32 esp_id  = 1;
  uint32 src_esp = 2;
  uint32 src_led = 3;
  uint32 seq     = 4;
  uint64 t_us    = 5;
```

```
}

message ExperimentConfig {
  uint32 num_esp      = 1;
  uint32 leds_per_esp = 2;
  float  epsilon      = 3;
  uint32 loss_pct     = 4;
  uint32 latency_ms   = 5;
  uint32 qos          = 6;
  uint32 seed         = 7;
}
```

Tópico	Quién publica	Quién recibe	Mensaje
exp/esp/{esp}/led/{led}/flash	ESP	ESP vecinos, backend	Flash
exp/esp/{esp}/led/{led}/phase	ESP	backend	PhaseReport
exp/esp/{esp}/rx	ESP	backend	RxEvent
exp/ctl/config	backend	todos los ESP	ExperimentConfig

Cada tópico es un canal de MQTT:

- **flash:** cada disparo de un LED. Los vecinos lo usan para ajustar su fase.
- **phase:** fase y estado de un LED, para el frontend.
- **rx:** confirmación de que un nodo recibió un flash, para medir la entrega.
- **config:** parámetros de la corrida, enviados por el backend.

En los tópicos, {esp} y {led} son números. Por ejemplo, exp/esp/3/led/2/flash es el disparo del LED 2 de la ESP 3.

El tópico `rx` es tráfico adicional que no existiría sin el experimento. Se mide como overhead y se reporta por separado.

Frontend

El frontend tiene cuatro paneles:

1. **Nodos:** una tarjeta por ESP con sus LEDs. Cada LED muestra encendido o apagado, una barra de fase de 0 a 2π, y se ilumina al disparar.
2. **Red:** los nodos dispuestos en círculo. Cada flash se dibuja como una línea animada del emisor al broker y del broker a cada receptor.
3. **Métricas en vivo:** gráfico de r(t), mensajes por segundo, pérdidas acumuladas y latencia reciente.

4. **Controles:** deslizadores para ε, pérdida, latencia y QoS. Cada cambio inicia una nueva corrida con esa configuración.

Configuraciones y repeticiones

Variamos un factor a la vez respecto de una configuración base: 8 ESP, 4 LEDs por ESP (32 osciladores), ε fijo, pérdida 0 %, latencia 0 ms y QoS 1.

Factor	Valores a probar	Pregunta que responde
Nodos ESP	2, 4, 8	¿Cómo escala el tiempo de convergencia?
LEDs por ESP	1, 2, 4	¿Afecta la densidad de osciladores por nodo?
Acoplamiento ε	bajo, medio, alto (a definir)	¿Hay un umbral para sincronizar?
Pérdida	0 %, 1 %, 5 %	¿Cuánto se degrada la convergencia?
Latencia	0 ms, 50 ms, 200 ms	¿Cómo afecta el tiempo de propagación?
QoS	0 y 1	¿Cuánto pesa la garantía de entrega?

Cada configuración se ejecuta 5 veces con semillas fijas, y se reportan media y desvío.

Métricas

Cada métrica responde a una pregunta de la monografía. Las de convergencia se calculan sobre r(t); las de costo, sobre cada mensaje publicado.

Métrica	Cómo se calcula	Qué permite concluir
Tiempo de convergencia	Primer instante con r ≥ 0,95	Velocidad de sincronización
r final	Promedio de r en los últimos 10 s	Si la red queda sincronizada
Bytes por flash (protobuf)	Tamaño del mensaje con cabeceras MQTT	Costo de ancho de banda
Bytes por flash (JSON)	Mismo mensaje en JSON	Ahorro de la serialización
Mensajes por segundo	Total de publicaciones / tiempo	Carga de la red
Tasa de entrega	Recibidos / enviados por par emisor-receptor	Efecto de la pérdida y del QoS
Latencia flash → recepción	RxEvent.t_us − Flash.t_us	Tiempo real de propagación

Métrica	Cómo se calcula	Qué permite concluir
Factor de tiempo real	Tiempo simulado / tiempo del host	Validez de la corrida

Dos cuidados: el factor de tiempo real se reporta en cada corrida, y por debajo de un umbral a definir la corrida se descarta. Las latencias se calculan con un reloj de referencia común o con los relojes simulados sincronizados antes de cada corrida.

Procedimiento de cada corrida

1. Levantar el broker Mosquitto.
2. Levantar el backend y el frontend.
3. Levantar las 8 instancias QEMU, cada una con su esp_id.
4. Enviar la configuración base desde el backend por exp/ctl/config.
5. Esperar la confirmación de cada nodo.
6. Ejecutar la corrida durante 120 s, o hasta que r ≥ 0,95 se mantenga 30 s.
7. Guardar mensajes y métricas en CSV o JSON, junto con la configuración usada.
8. Repetir con las semillas y variantes de la sección de configuraciones.

Limitaciones

- QEMU no emula WiFi. La red entre ESP es Ethernet emulado o el stack de red de QEMU, y lo inalámbrico queda fuera.
- La emulación es por software, así que el timing no es idéntico al de un ESP32 físico. Por eso se controla el factor de tiempo real.
- La CPU del host limita el número de instancias más que la RAM. Por eso el máximo es 8 nodos.
- El tráfico del tópico rx y de los reportes de fase se reporta por separado del costo del algoritmo.
- El frontend, el backend y el broker comparten recursos con las instancias. Conviene cerrar aplicaciones innecesarias durante las corridas.

Hipótesis orientativas

- Para ε por encima de un umbral y sin pérdida, la red converge a r ≥ 0,95.
- Aumentar la latencia retrasa la convergencia, pero no impide llegar a la sincronía.
- Con 5 % de pérdida, la convergencia se degrada, y QoS 1 debería compensar parte de la pérdida a costa de más mensajes.
- El número de mensajes por segundo crece aproximadamente de forma lineal con N, y el tiempo de convergencia depende de la topología y de ε.

Estas hipótesis sirven tanto para confirmarse como para descartarse.